Web-Technologien UE - Assignment 8

Raphael Bleier, WS 2026 - Angular 22.2

Exercise 1 - Angular Lernzettel

Meine Zusammenfassung zu den Guides auf angular.dev. Die Beispiele sind aus der Homes-App von Exercise 2, damit ich beim Präsentieren direkt im Code zeigen kann, wo was ist.

Grundaufbau

Eine Angular-App ist ein Baum aus Komponenten. `main.ts` startet die App mit `bootstrapApplication(App, appConfig)`, und `App` ist die Wurzel-Komponente. In `app.config.ts` stehen die globalen Provider, z.B. der Router. Seit ein paar Versionen sind alle Komponenten standalone, NgModules braucht man nicht mehr. Jede Komponente importiert selbst, was sie im Template verwendet. Geschrieben wird in TypeScript, die CLI (`ng new`, `ng serve`, `ng build`) baut daraus JavaScript für den Browser.

```
src/
  main.ts              startet die app
  index.html           enthält nur <app-root>
  app/
    app.ts             wurzel-komponente mit header und <router-outlet>
    app.config.ts      provider (router)
    app.routes.ts      welche url zeigt welche komponente
    home/              liste + suche
    details/           detailseite + formular
    housing-location/  eine karte in der liste
    housing.service.ts holt die daten vom server
    housinglocation.ts interface für die daten
```

Components

Eine Komponente ist eine TypeScript-Klasse mit dem `@Component` Decorator. Der legt fest: `selector` (der HTML-Tag, z.B. `app-housing-location`), `template` oder `templateUrl`, `styleUrls` (CSS gilt nur für diese Komponente) und `imports` (andere Komponenten oder Direktiven, die im Template vorkommen). Die Klasse hält die Daten und Methoden, das Template zeigt sie an.

Daten von der Eltern-Komponente kommen über Inputs. In der App: `housingLocation = input.required<HousingLocationInfo>()` in HousingLocation, und Home übergibt sie mit `<app-housing-location [housingLocation]="housingLocation" />`. `input()` gibt ein Signal zurück, deshalb steht im Template `housingLocation().name` mit Klammern. Die andere Richtung (Kind meldet etwas an Eltern) geht mit `output()` und Events.

Templates

Das ist HTML mit ein paar Extras:

`{{ housingLocation?.name }}` ist Interpolation, schreibt einen Wert als Text rein.
`[src]="housingLocation?.photo"` ist Property Binding, setzt eine DOM-Property auf einen Ausdruck.
`(click)="filterResults(filter.value)"` ist Event Binding, ruft bei einem Event eine Methode auf.

`#filter` ist eine Template-Variable, damit kommt man im Template direkt an das input-Element. `@for (item of liste; track $index) { ... }` wiederholt einen Block für jedes Element, `@if (bedingung) { ... } @else { ... }` zeigt etwas nur unter einer Bedingung an. Das ist die neue Control-Flow-Syntax, früher hat man dafür `*ngFor` und `*ngIf` verwendet. `track` sagt Angular, woran es Elemente wiedererkennt, damit nicht bei jeder Änderung alles neu gebaut wird.

Dependency Injection

Statt dass eine Komponente ihre Abhängigkeiten selbst mit `new` erzeugt, bekommt sie sie von Angular. In Angular 22 markiert man einen Service mit `@Service()`, er wird dann automatisch app-weit als eine einzige Instanz (Singleton) bereitgestellt. Früher schrieb man dafür `@Injectable({ providedIn: 'root' })`. In der Komponente holt man ihn mit `housingService = inject(HousingService)`.

Warum: Die Datenlogik ist an einer Stelle und alle Komponenten teilen sich dieselbe Instanz. Beim Testen kann man den Service leicht durch eine Fake-Version ersetzen. Home und Details wissen nicht, ob die Daten aus einem Array, von json-server oder einer echten API kommen, das weiß nur der Service. Genau das hat man bei Schritt 14 im Tutorial gemerkt: Nur der Service wurde auf fetch umgebaut, die Komponenten sind fast gleich geblieben.

Routing

In `app.routes.ts` steht ein Array von Routen: `{ path: '', component: Home }` und `{ path: 'details/:id', component: Details }`. `provideRouter(routes)` in der Config aktiviert den Router. `<router-outlet />` in App ist der Platz, an dem die aktuelle Seite angezeigt wird. Verlinkt wird mit `[routerLink]="['/details', housingLocation().id]"` statt mit href. Dann lädt der Browser die Seite nicht neu, Angular tauscht nur die Komponente aus (Single Page Application). Den Parameter liest Details mit `inject(ActivatedRoute).snapshot.params['id']`. `title` in der Route setzt den Tab-Titel.

Forms

Angular hat zwei Arten. **Reactive Forms** wie in der App: Das Formular wird im TypeScript als `FormGroup` mit `FormControl`s gebaut und im Template mit `[formGroup]="applyForm"` und `formControlName="firstName"` verbunden. Den Wert liest man mit `applyForm.value`. Validierung geht über Validators (z.B. `Validators.required`, `Validators.email`). **Template-driven Forms** mit `[(ngModel)]` sind einfacher für kleine Formulare, aber schwerer zu testen. Neu dazugekommen sind Signal Forms (`@angular/forms/signals`), da ist das Formular-Modell selbst ein Signal.

HTTP Client und Observables

Im Tutorial holt der Service die Daten mit dem normalen `fetch` und Promises (async/await). Der Angular-Weg ist der `HttpClient`: `provideHttpClient()` in der Config, dann `http = inject(HttpClient)` und `http.get<HousingLocationInfo[]>(url)`. Der gibt ein Observable zurück, kein Promise.

Ein Observable (aus RxJS) ist ein Datenstrom, der mehrere Werte über die Zeit liefern kann. Ein Promise liefert genau einen. Ein Observable ist außerdem lazy: Der Request geht erst raus, wenn jemand `.subscribe(...)` aufruft oder es im Template mit der `async` Pipe verwendet wird. Mit Operatoren wie `map`, `filter` und `catchError` kann man den Strom umbauen. Vorteile vom HttpClient gegenüber fetch: JSON wird automatisch geparst, man kann Interceptors verwenden (z.B. den JWT-Token aus Blatt 7 automatisch in jeden Request hängen), Requests lassen sich abbrechen und Tests gehen einfacher.

Signals

Ein Signal ist ein Wert, von dem Angular weiß, wer ihn verwendet. `count = signal(0)`, lesen mit `count()`, ändern mit `count.set(5)` oder `count.update(c => c + 1)`. `computed(() => ...)` ist ein abgeleiteter Wert, der sich automatisch neu berechnet. `effect(() => ...)` führt Code aus, wenn sich ein verwendetes Signal ändert. Wenn ein Signal sich ändert, aktualisiert Angular gezielt nur die Stellen im Template, die es verwenden.

In der App sieht man den Unterschied ganz gut. Home und Details speichern die Daten in normalen Properties und müssen nach dem Laden `changeDetectorRef.markForCheck()` aufrufen, damit Angular merkt, dass sich was geändert hat. Mit `housingLocationList = signal<HousingLocationInfo[]>([])` und `.set(...)` wäre das automatisch. Inputs sind über `input()` schon Signals.

Exercise 2 - Erste lokale Angular App

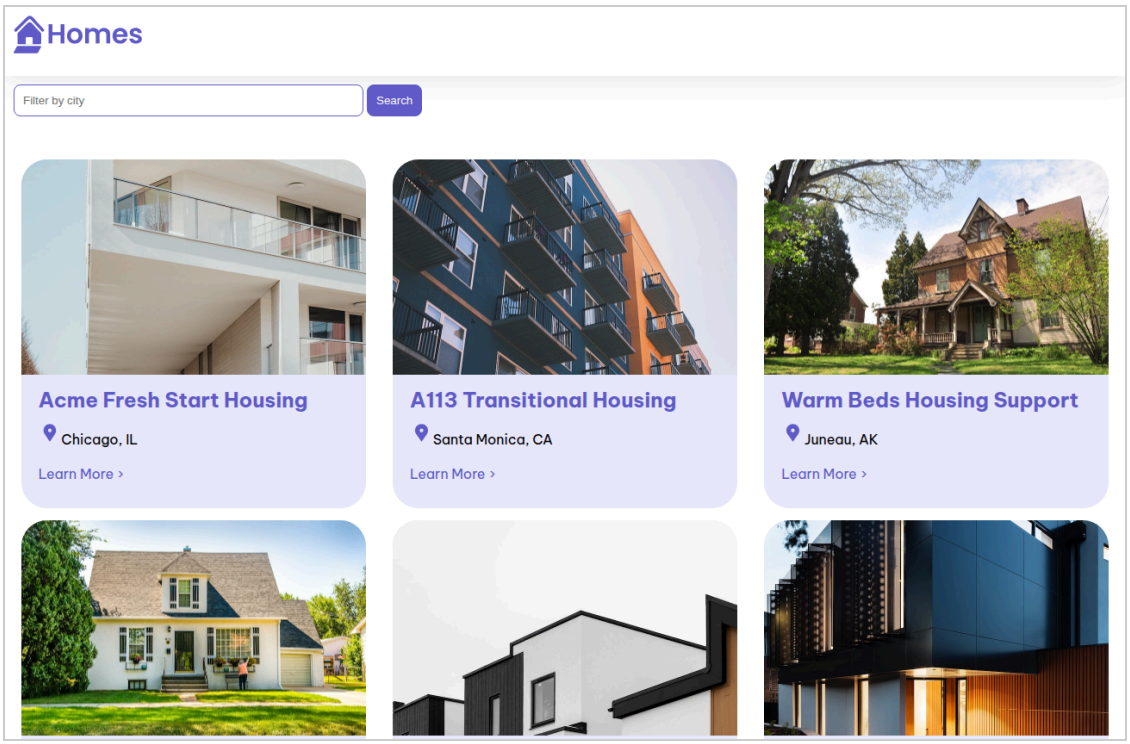
Gemacht nach dem offiziellen Tutorial "Your first Angular app" (angular.dev/tutorials/first-app), alle 14 Schritte: Komponenten Home, HousingLocation und Details, das Interface, Inputs und Property Binding, `@for`, ein Service mit DI, Routing mit Detailseite, ein Reactive Form, die Suche nach Stadt und am Ende die Daten über HTTP von json-server.

Unterschiede zum Tutorial: Ich hab ein echtes CLI-Projekt mit `ng new` angelegt (Angular 22.2), die Routen liegen deshalb in `app.routes.ts` und werden über `app.config.ts` geladen statt direkt in main.ts. Die Bilder liegen im `public` Ordner, deshalb heißt der Pfad `/logo.svg` statt `/public/logo.svg`. Den unbenutzten Import von Home in app.ts hab ich rausgenommen, sonst kommt beim Build die Warnung NG8113.

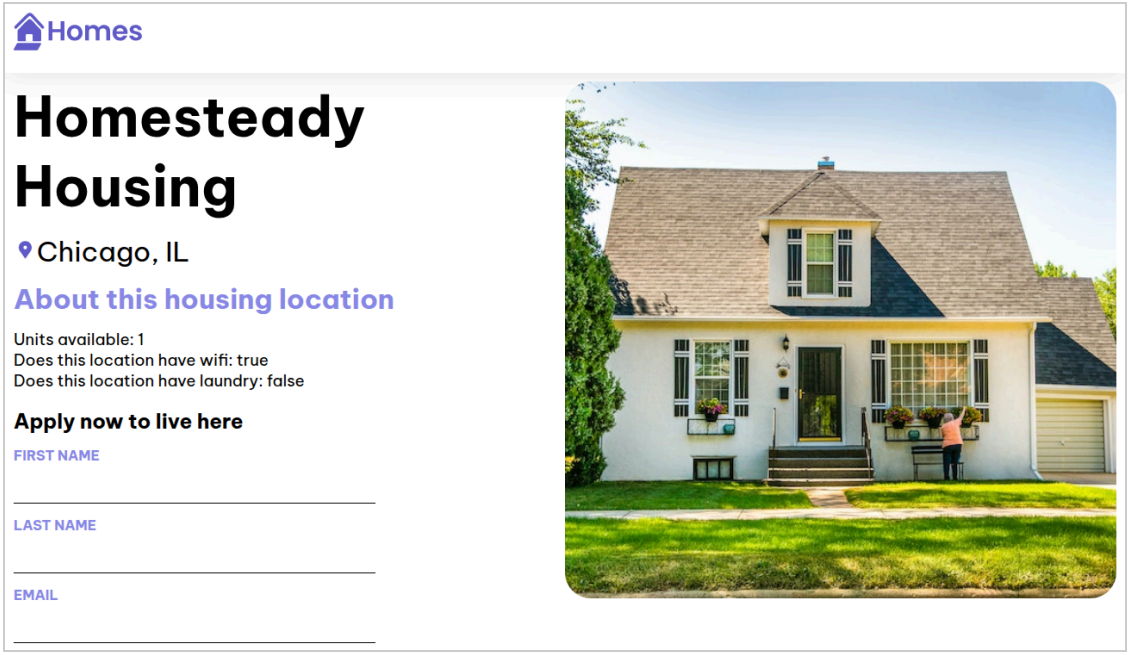
Starten

```
cd homes-app
npm install
npm run api      # json-server auf http://localhost:3000/locations
npm start        # in einem zweiten terminal, app auf http://localhost:4200
```

Angular CLI 22 braucht Node 22.22.3 oder neuer. Bei mir war über apt 22.22.1 installiert, deshalb hab ich Node 24 LTS lokal in ~/.local/node installiert und mit `export PATH=$HOME/.local/node/bin:$PATH` verwendet.



Startseite mit allen Wohnungen von json-server



Detailseite /details/3 mit dem Bewerbungsformular